PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Функции: разбиение программы на части

Содержание

- Вступление
- Зачем нужны функции
- Синтаксис функции
- Параметры и аргументы
- Оператор return
- Локальные переменные
- Функции и обработка данных
- Памятка
- Итоги урока
- Материалы к занятию

Вступление

Представь: ты написал программу для подсчета площади круга. Потом тебе понадобилось считать ее три раза – в разных местах кода. Копировать одни и те же строки снова и снова?
Функция – это способ дать имя блоку кода и вызывать его столько раз, сколько нужно. Один раз описал – используй везде. Именно так написаны все серьезные программы: они состоят из функций.
В рабочих задачах это выглядит так: одна функция проверяет заявку, другая считает стоимость заказа, третья формирует строку отчёта. Каждую можно вызвать для любой заявки или заказа, не переписывая код.
Сегодня мы научимся создавать собственные функции, передавать им данные через параметры и получать результат через `return` – и напишем программу, в которой повторяющиеся действия вынесены в отдельные функции.
После этого занятия вы сможете:

- определять функцию с помощью `def`;

- вызывать функцию и передавать ей аргументы;
- возвращать результат через `return`;
- использовать параметры по умолчанию;
- разбивать программу на логические части – функции.

Зачем нужны функции

Без функций	С функциями
Один и тот же код копируется несколько раз	Код написан один раз, вызывается многократно
При ошибке нужно исправлять в каждом месте	Исправление в одном месте – работает везде
Сложно понять, что делает длинная программа	Каждая функция – одна четко названная задача
Невозможно повторно использовать код	Функцию можно использовать в других программах

Синтаксис функции

```
def имя_функции(параметр1, параметр2):
    """Документация функции (необязательно)"""
    # тело функции
    return результат # необязательно
```

```
# Пример: функция приветствия
def greet(name):
    print(f"Привет, {name}!")

# Вызов
greet("Аня") # Привет, Аня!
greet("Боря") # Привет, Боря!
```

- `def` – ключевое слово для определения функции.
- Двоеточие и отступ 4 пробела – обязательны, как в `if` и `for`.
- Функцию нужно ОПРЕДЕЛИТЬ (написать `def`) до того, как ее ВЫЗВАТЬ.

Строка в тройных кавычках сразу после `def` — **документация** функции. Python не выполняет её как команду: при вызове функции эта строка ничего не делает, но сохраняется вместе с функцией. Её видят коллеги и среды разработки: по ней понятно, что делает функция, не читая её код.
Посмотреть документацию функции можно двумя способами: командой `help(имя_функции)` или через атрибут `имя_функции.__doc__`. Так же работает справка и для встроенных функций, например `help(print)`.

```
def area(r):
    """Возвращает площадь круга радиуса r."""
    return 3.14 * r ** 2

print(area.__doc__) # Возвращает площадь круга радиуса r.
help(area)          # справка по функции, включая её документацию
```

Параметры и аргументы

Термин	Определение	Пример
Параметр	Переменная в определении функции	`def greet(name):` ← `name`
Аргумент	Значение, передаваемое при вызове	`greet("Аня")` ← `"Аня"`
По умолчанию	Значение параметра, если не передан аргумент	`def greet(name="друг"):`

```
def power(base, exp=2): # exp по умолчанию = 2
    return base ** exp

print(power(3)) # 9 – exp = 2 (по умолчанию)
print(power(3, 3)) # 27 – exp = 3 (передан явно)
print(power(2, 10)) # 1024
```

Порядок параметров. Параметры со значением по умолчанию записываются *после* обязательных параметров. Это правило синтаксиса Python: обязательный параметр нельзя

записывать после параметра со значением по умолчанию. Если нарушить порядок, Python сообщит об ошибке `SyntaxError` ещё до запуска программы:

```
# Ошибка: SyntaxError: non-default argument follows default argument
# def power(exp=2, base):
#     return base ** exp

# Правильно: сначала обязательный параметр, затем параметр по умолчанию
def power(base, exp=2):
    return base ** exp
```

Параметр — как поле в бланке заявки, а аргумент — то, что в это поле вписали для конкретного случая. Бланк один, заполнений может быть сколько угодно.

Оператор return

`return` завершает выполнение функции и возвращает значение вызывающему коду.

```
def add(a, b):
    return a + b # возвращает сумму

result = add(5, 3) # result = 8
print(add(10, 20)) # 30
print(add(1, 2) + add(3, 4)) # 10
```

Ситуация	Что вернет функция
`return` отсутствует	`None` — специальное значение «нет значения»
`return` без значения	`None`
`return` значение	указанное значение
`return val1, val2`	кортеж `(val1, val2)`

Важно различать `print` и `return`. `print` только показывает значение на экране, а `return` передаёт его дальше в программу — его можно сохранить в переменную, сложить, записать в отчёт.
Разница хорошо видна на примере двух функций, которые считают одно и то же:

```
def show_sum(a, b):
    print(a + b)   # только выводит результат на экран

def get_sum(a, b):
    return a + b   # возвращает результат в программу

x = show_sum(2, 3) # на экран выводится 5
y = get_sum(2, 3)  # на экран ничего не выводится
print(x)           # None — show_sum ничего не вернула
print(y)           # 5
print(get_sum(2, 3) * 10) # 50 — возвращённое значение можно использовать дальше
```

Если функция возвращает несколько значений через запятую, результатом будет кортеж. Его можно сразу **распаковать** в несколько переменных — их количество слева должно совпадать с количеством возвращаемых значений:

```
def min_max(numbers):
    return min(numbers), max(numbers)

result = min_max([4, 9, 1, 7])
print(result)    # (1, 9) — кортеж

low, high = min_max([4, 9, 1, 7])  # распаковка
print(low, high) # 1 9
```

Локальные переменные

Переменные, которым значение присваивается внутри функции, — **локальные**: они создаются при каждом вызове функции и недоступны снаружи. Параметры функции тоже являются её локальными переменными.

```
def calculate(x):
    result = x * 2 # локальная переменная
    return result

print(calculate(5)) # 10
# print(result) # ← NameError: result не существует снаружи
```

Поэтому результат работы функции передают наружу через `return`, а не через переменную внутри неё.
При этом функция может **читать** переменные, созданные вне её (глобальные). Но если внутри функции присвоить значение переменной с тем же именем, Python создаст новую локальную переменную, а внешняя не изменится:

```
rate = 10 # глобальная переменная

def price_with_rate(price):
    return price + rate # чтение глобальной переменной — допустимо

def change_rate():
    rate = 20 # новая локальная переменная, глобальная rate не меняется

change_rate()
print(price_with_rate(100)) # 110
print(rate)                 # 10
```

Изменить глобальную переменную изнутри функции можно с помощью слова `global`, но так делать не рекомендуется: программу становится трудно проверять. Надёжнее передавать данные через параметры и возвращать результат через `return`.

Функции и обработка данных

Функции – основа любой обработки данных. Классификатор из прошлого урока можно вынести в функцию:

> **Важно**

> Здесь «классификатор» — это набор правил `if` / `elif`, которые человек придумал и записал сам. Это не обучаемая ML-модель: модель машинного обучения не получает готовые правила, а подбирает зависимость по примерам из данных.

```
def classify(score, attendance, project):
    if score >= 85 and attendance >= 80 and project:
        return "Отличник"
    elif score >= 70 and project:
        return "Хорошист"
    elif score >= 60:
        return "Удовл."
    else:
        return "Неудовл."

# Применяем к каждому объекту датасета
dataset = [
    {"name":"Аня","score":92,"attendance":95,"project":True},
    {"name":"Боря","score":58,"attendance":70,"project":False},
]
for row in dataset:
    row["class"] = classify(row["score"], row["attendance"], row["project"])
    print(row["name"], "→", row["class"])
```

Результат выполнения:

```
Аня → Отличник
Боря → Неудовл.
```

Чтобы этот код работал, в каждой записи датасета должны быть ключи `"name"`, `"score"`, `"attendance"` и `"project"`, а значения — подходящего типа: `name` — строка, `score` и `attendance` — числа, `project` — `True` или `False`. Если в какой-то записи ключа нет, обращение `row["score"]` вызовет ошибку `KeyError`.
Так же устроена обработка рабочих данных: функция получает одну запись — заявку, заказ, позицию на складе — и возвращает для неё метку или результат расчёта. Цикл применяет эту функцию ко всем записям таблицы.

Памятка

Если нужно…	Используйте…
Определить функцию	`def name(params):`
Вернуть значение	`return value`
Значение по умолчанию	`def f(x, y=10):` (сначала обязательные параметры)
Вызвать функцию	`result = name(args)`
Несколько возвращаемых	`return val1, val2`; распаковка: `a, b = f()`
Принять несколько аргументов	`def f(a, b, c):`
Документировать функцию	`"""Описание"""` сразу после `def`; посмотреть: `help(f)`

Итоги урока

На этом занятии вы:

- научились определять функции с помощью `def`;
- передавали данные в функцию через параметры и получали результат через `return`;
- использовали параметры по умолчанию;
- разбивали программу на логические части;
- применяли функции к обработке данных и датасетов.

Функции – это «кирпичики», из которых складывается программа: программист пишет функции, а потом комбинирует их для решения сложных задач. Библиотеки Python, например pandas и scikit-learn, тоже дают готовые функции, которые можно вызывать в своих программах.

Материалы к занятию

- 110.pptx (14.7 МБ)